Table des matières

Labo 06 — Orchestrer et rattraper ..... 1
Avant de commencer ..... 2
Étape 1 — Régénérer les sources jusqu'à mars 2026 ..... 3
Étape 2 — Écrire la fenêtre de temps du pipeline ..... 3
Étape 3 — Brancher la fenêtre et le partage des livraisons sur les étapes ..... 5
Étape 4 — Charger une seule journée, et voir les garde-fous répondre ..... 20
Étape 5 — Tester le rattrapage sans Airflow ..... 21
Étape 6 — Allumer Airflow et lire les fichiers qui le composent ..... 26
Étape 7 — Vérifier le pool de l'entrepôt ..... 26
Étape 8 — Écrire le DAG quotidien et lancer une journée ..... 26
Étape 9 — Préparer un témoin, puis laisser le DAG mensuel rattraper ..... 30
Étape 10 — Prouver que le rattrapage égale le chargement d'un bloc ..... 32
Étape 11 — Versionner la fenêtre, les DAG et les tests ..... 35
Dépannage ..... 35

# Labo 06 — Orchestrer et rattraper

*Labo · Cours 6/12 — Formation Data Engineering*

But : prolonger les sources, qui s'arrêtaient au 31 décembre 2025, de trois mois de 2026 (janvier à mars), les mois que le DAG mensuel rattrapera ; donner au pipeline du Cours 05 une tranche de temps ; monter l'Airflow de la plateforme et lire ce qui le compose ; écrire deux DAG et laisser le mensuel rattraper les trois mois ; prouver que ce rattrapage égale un chargement d'un seul bloc ; garder le tout dans le dépôt. Le pourquoi de chaque étape est dans `Theorie_06_DE.md`, section 9 et annexe A.

Le labo se fait en deux séances, coupées au point d'arrêt de l'étape 5 : la première se passe d'Airflow.

Trois fenêtres. Chacune reste dans son dossier du début à la fin du labo : aucun `cd` à taper après l'ouverture.

| **Fenêtre** | **Dossier** | **Ce qu'on y fait** |
|---|---|---|
| 1. Anaconda Prompt, `formations` actif | `Projet_Boreal_intermediaire` | le Bloc-notes, Python, `pytest`, Git, le générateur |



© 2026 iAlexMG. Tous droits réservés.

| Fenêtre | Dossier | Ce qu'on y fait |
|---|---|---|
| 2. Anaconda Prompt devenue Ubuntu | `Projet_Boreal_intermediaire\plateforme` | Docker, et Airflow par sa ligne de commande |
| 3. Anaconda Prompt, à l'étape 10 | `Projet_Boreal_intermediaire` | l'interface web de DuckDB (`duckdb -ui`) |

À partir de l'étape 6, le navigateur sert aussi : l'interface d'Airflow répond sur `http://localhost:8080`.

Liste de contrôle :

- ☐ Sources régénérées jusqu'à 2026-03 : 1 989 fichiers, empreinte `2081493759980946…`
- ☐ `fenetre.py` écrit, branché sur `raw` et sur la porte d'entrée, avec `--livraisons`
- ☐ Une journée de télémétrie chargée seule, 222 lignes, puis les trois garde-fous
- ☐ `pytest` : `5 passed`
- ☐ Pile d'Airflow construite et démarrée, cinq services sains ; pool `entrepot` à une place
- ☐ `boreal_quotidien` inscrit et lancé sur le 2026-03-15
- ☐ Témoin chargé d'un bloc : 42 fichiers, 20 664 lignes
- ☐ `boreal_mensuel` : trois exécutions vertes, une par mois
- ☐ Comparaison : 14 tables, 0 écart
- ☐ Code validé dans `01_de`, `git status` propre

---

## Avant de commencer

Fenêtre 1, une Anaconda Prompt, dans `Projet_Boreal_intermediaire` :

```
conda activate formations
REM Attendu : (formations) au début de l'invite.
set PYTHONPATH=01_de
REM Attendu : rien. Python cherchera aussi ses paquets dans 01_de.
git -C 01_de status
REM Attendu : nothing to commit, working tree clean.
```



© 2026 iAlexMG. Tous droits réservés.

```
python -m pipeline --etapes
REM Attendu : la liste « Étapes du pipeline » : bronze, raw, corpus, erp et meteo.
```

Une étape du pipeline est une de ses commandes, `python -m pipeline raw` par exemple, et non une étape d'un labo : le Labo 05 en a neuf, le pipeline en a cinq. Si un résultat diffère, finir d'abord le Labo 05.

Fenêtre 2 : une nouvelle Anaconda Prompt, dans `Projet_Boreal_intermediaire`, puis :

```
cd plateforme
wsl
```

Attendu : une invite qui finit par `…/Projet_Boreal_intermediaire/plateforme$`. Cette fenêtre est désormais Ubuntu, dans le dossier de la plateforme, et y reste. La garder ouverte jusqu'à la fin du labo.

Construire l'image d'Airflow, puis démarrer ses services. Airflow ne sert qu'à partir de l'étape 6 : qui fait le labo en deux séances peut garder ce bloc pour le début de la seconde. Fenêtre 2 :

```
docker compose build airflow-init
# Attendu : naming to docker.io/library/boreal-airflow. La première fois, l'image officielle
# d'Airflow se télécharge, 3,18 Go ; ensuite, quelques secondes.
docker compose up -d airflow-apiserver airflow-scheduler airflow-dag-processor airflow-triggerer
# Attendu, en moins d'une minute : la première fois, l'image postgres:16 se télécharge ; puis
# airflow-init Exited, et les quatre services Started.
docker compose ps
# Attendu : airflow-apiserver, airflow-dag-processor, airflow-db, airflow-scheduler et
# airflow-triggerer, chacun (healthy), et erp, Up. Encore (health: starting) : relancer ps.
```

## Étape 1 — Régénérer les sources jusqu'à mars 2026

*Théorie : `Theorie_06_DE.md`, annexe A.1.*

1.1 Régénérer les sources avec trois mois de plus. Celles des Cours 01 à 05 s'arrêtaient au 31 décembre 2025 ; `--jusqua 2026-03` ajoute janvier, février et mars 2026, comme si le temps avait passé depuis. Fenêtre 1 :

```
python ..\_univers\boreal\generer_boreal.py --projet . --jusqua 2026-03
REM Attendu : le bilan des sources, puis 1 989 fichiers, 177,5 Mo, empreinte globale 2081493759980946...
```

## Étape 2 — Écrire la fenêtre de temps du pipeline

*Théorie : `Theorie_06_DE.md`, annexe A.2.*

2.1 Écrire le module de la fenêtre. Fenêtre 1 :



© 2026 iAlexMG. Tous droits réservés.

```
notepad 01_de\pipeline\fenetre.py
```

Accepter la création. Coller, enregistrer, fermer :

```
# -*- coding: utf-8 -*-
"""fenetre.py : la tranche de temps qu'une exécution du pipeline traite.

Un lot est ce que le Cours 03 a écrit dans la colonne _lot de la RAW : le mois d'un extrait
(2026-01), le jour d'une archive de télémétrie (2026-01-14), ou « complet » pour un export livré
en entier. La fenêtre dit quels fichiers regarder ; le journal des chargements dit lesquels ont
déjà été chargés. Les deux se complètent, et c'est le journal qui garde l'idempotence quand deux
fenêtres se chevauchent.

Un export livré en entier entre dans toutes les fenêtres : il n'a pas de date, et son empreinte
suffit à décider s'il a bougé. Un lot que la fenêtre ne sait pas lire y entre aussi : une tranche
de temps ne doit jamais faire disparaître une livraison en silence.
"""
from calendar import monthrange
from dataclasses import dataclass
from datetime import date


class DateIllisible(ValueError):
    """Une borne de la ligne de commande n'est ni AAAA-MM ni AAAA-MM-JJ."""


def _bornes(texte: str) -> tuple:
    """La première et la dernière journée que « 2026-01 » ou « 2026-01-14 » couvre."""
    morceaux = texte.split("-")
    try:
        if len(morceaux) == 2:
            annee, mois = int(morceaux[0]), int(morceaux[1])
            return date(annee, mois, 1), date(annee, mois, monthrange(annee, mois)[1])
        if len(morceaux) == 3:
            jour = date(int(morceaux[0]), int(morceaux[1]), int(morceaux[2]))
            return jour, jour
    except ValueError as erreur:
        raise DateIllisible(f"« {texte} » n'est pas une date : {erreur}") from erreur
    raise DateIllisible(f"« {texte} » n'est ni AAAA-MM ni AAAA-MM-JJ.")


@dataclass(frozen=True)
class Fenetre:
```



© 2026 iAlexMG. Tous droits réservés.

```
    """La tranche traitée par une exécution. Sans borne, elle contient tout."""

    depuis: date = None
    jusqua: date = None

    @classmethod
    def depuis_la_ligne_de_commande(cls, depuis: str = None, jusqua: str = None) -> "Fenetre":
        """« --depuis 2026-01 --jusqua 2026-01 » donne le mois de janvier en entier."""
        return cls(depuis=_bornes(depuis)[0] if depuis else None,
                   jusqua=_bornes(jusqua)[1] if jusqua else None)

    @property
    def ouverte(self) -> bool:
        """Vrai quand aucune borne n'a été donnée : l'étape travaille comme au Cours 05."""
        return self.depuis is None and self.jusqua is None

    def contient(self, lot: str) -> bool:
        """Le lot d'un fichier tombe-t-il dans la tranche ?

        Un lot relivré garde sa date : 2023-03_bis reste le mois de mars 2023.
        """
        if self.ouverte:
            return True
        try:
            debut, fin = _bornes(lot.split("_")[0])
        except DateIllisible:
            return True
        return not (self.jusqua is not None and debut > self.jusqua
                    or self.depuis is not None and fin < self.depuis)

    def __str__(self) -> str:
        if self.ouverte:
            return "tout l'historique"
        return f"du {self.depuis or 'début'} au {self.jusqua or 'dernier jour livré'}"
```

## Étape 3 — Brancher la fenêtre et le partage des livraisons sur les étapes

*Théorie : `Theorie_06_DE.md`, annexe A.3.*

Les six fichiers de cette étape existent déjà. Pour chacun : tout sélectionner dans le Bloc-notes (Ctrl+A), coller la nouvelle version à sa place, enregistrer, fermer.

3.1 Le module de la `RAW`, qui apprend la fenêtre et les livraisons. Fenêtre 1 :



© 2026 iAlexMG. Tous droits réservés.

```
notepad 01_de\pipeline\raw.py
```

```
# -*- coding: utf-8 -*-
"""raw.py : les 22 livraisons de Boréal chargées dans la couche RAW (Cours 03, empaqueté au Cours 05).

Ce qu'il fait. Pour chacune des livraisons de LIVRAISONS, il cherche les fichiers dans sources/,
calcule l'empreinte SHA-256 de chacun, et ne charge que ceux que le journal des chargements ne
connaît pas encore, ou connaît avec une autre empreinte. Chaque fichier entre tel quel dans sa table
de l'entrepôt : toutes les colonnes en texte, un objet JSON dans une colonne JSON, et trois colonnes
de lignage, _fichier, _lot et _charge_le.

La règle qu'il illustre. L'idempotence : relancer ne double rien, parce que le journal se souvient de
chaque fichier chargé. Un fichier livré de nouveau sous le même nom, avec un autre contenu, remplace
ses propres lignes : c'est ainsi qu'un export complet se met à jour. Un fichier livré sous un autre
nom est une autre livraison : l'extrait 2023-03_bis passe, et la RAW le garde, marqué par son lot.

Ce que le Cours 05 a changé, par rapport au script du Cours 03. Le module ne se lance plus seul et
n'ouvre plus l'entrepôt : il reçoit la connexion de la porte d'entrée, écrit dans le journal plutôt
qu'à l'écran, et rend ses comptes à l'appelant, qui les inscrit dans pipeline.executions. Les chemins
viennent de la configuration. La logique de chargement, elle, n'a pas bougé d'une ligne.
"""
import hashlib
from datetime import datetime, timezone
from pathlib import Path

from .entrepot import deja_charge
from .fenetre import Fenetre
from .journal import LOG

# Une livraison : la table de la RAW, le motif de ses fichiers, leur forme, et d'où vient le lot.
#   dossier : le dossier du mois (2023-03, 2023-03_bis) ; jour : la date du nom de fichier ;
#   complet : un export livré en entier, qui remplace le précédent.
LIVRAISONS = [
    ("erp_t005u", "sources/erp/extraits/*/T005U.csv", "csv", "dossier"),
    ("erp_kna1", "sources/erp/extraits/*/KNA1.csv", "csv", "dossier"),
    ("erp_mara", "sources/erp/extraits/*/MARA.csv", "csv", "dossier"),
    ("erp_makt", "sources/erp/extraits/*/MAKT.csv", "csv", "dossier"),
    ("erp_vbak", "sources/erp/extraits/*/VBAK.csv", "csv", "dossier"),
    ("erp_vbap", "sources/erp/extraits/*/VBAP.csv", "csv", "dossier"),
    ("erp_cdhdr", "sources/erp/extraits/*/CDHDR.csv", "csv", "dossier"),
    ("erp_cdpos", "sources/erp/extraits/*/CDPOS.csv", "csv", "dossier"),
    ("erp_zserie", "sources/erp/extraits/*/ZSERIE.csv", "csv", "dossier"),
    ("garanties_clients", "sources/garanties/*/clients.csv", "csv", "dossier"),
```



© 2026 iAlexMG. Tous droits réservés.

```
    ("garanties_enregistrements", "sources/garanties/*/enregistrements.csv", "csv", "dossier"),
    ("garanties_visites", "sources/garanties/*/visites.csv", "csv", "dossier"),
    ("garanties_consentements", "sources/garanties/*/consentements.csv", "csv", "dossier"),
    ("garanties_reclamations", "sources/garanties/*/reclamations.json", "json", "dossier"),
    ("usine_actifs", "sources/usine/referentiel/actifs.csv", "csv", "complet"),
    ("usine_etiquettes", "sources/usine/referentiel/etiquettes.csv", "csv", "complet"),
    ("usine_bons_travail", "sources/usine/gmao/bons_travail.csv", "csv", "complet"),
    ("meteo_stations", "sources/meteo/stations.csv", "csv", "complet"),
    ("meteo_quotidien", "sources/meteo/quotidien.csv", "csv", "complet"),
    ("meteo_horaire", "sources/meteo/horaire_7060400.csv", "csv", "complet"),
    ("telemetrie_sorties", "sources/telemetrie/sorties_*.jsonl", "jsonl", "jour"),
    ("corpus_index", "sources/corpus/index.csv", "csv", "complet"),
]

# La requête qui lit une liste de fichiers, selon leur forme. Chacune rend la colonne filename.
LECTEURS = {
    "csv": "SELECT * FROM read_csv($fichiers, header = true, all_varchar = true, filename = true)",
    "json": ("SELECT json AS objet, filename "
             "FROM read_json_objects($fichiers, format = 'array', filename = true)"),
    "jsonl": ("SELECT sujet, recu_le, qos, message, filename "
              "FROM read_json($fichiers, format = 'newline_delimited', filename = true, "
              "columns = {'sujet': 'VARCHAR', 'recu_le': 'VARCHAR', 'qos': 'VARCHAR', 'message': 'JSON'})"),
}


def lot_du(fichier: str, regle: str) -> str:
    """Le lot d'un fichier : son dossier, le jour de son nom, ou « complet »."""
    chemin = Path(fichier)
    if regle == "dossier":
        return chemin.parent.name
    if regle == "jour":
        return chemin.stem.replace("sorties_", "")
    return "complet"


def empreinte(fichier: str) -> str:
    """L'empreinte SHA-256 du contenu : deux fichiers identiques la partagent, un octet la change."""
    return hashlib.sha256(Path(fichier).read_bytes()).hexdigest()


def a_charger(con, table: str, motif: str, regle: str, fenetre: Fenetre = None) -> list:
    """Les fichiers du motif que le journal ne connaît pas, ou connaît sous une autre empreinte.
```



© 2026 iAlexMG. Tous droits réservés.

```
    Les chemins se lisent depuis Projet_Boreal_intermediaire, où la porte d'entrée s'est placée : le lignage de la
    RAW s'écrit alors de la même façon sous Windows et sous Linux.

    Deux filtres, dans cet ordre, et l'ordre compte (Cours 06). La fenêtre écarte d'abord les
    fichiers dont le lot tombe hors de la tranche de cette exécution : elle ne lit rien, elle regarde
    des noms. Le journal écarte ensuite ceux qu'il a déjà chargés sous la même empreinte : lui doit
    lire chaque fichier restant pour en calculer l'empreinte. Filtrer par la fenêtre d'abord, c'est
    ne pas relire 1 572 fichiers pour en charger 14.
    """
    fenetre = fenetre or Fenetre()
    connus = deja_charge(con, table)
    fichiers = [p.as_posix() for p in Path(".").glob(motif)]
    dans_la_tranche = sorted(f for f in fichiers if fenetre.contient(lot_du(f, regle)))
    return [(f, sceau, lot_du(f, regle)) for f in dans_la_tranche
            if connus.get(f) != (sceau := empreinte(f))]


class LivraisonInconnue(ValueError):
    """Un préfixe de livraison qui n'existe pas, passé par --livraisons."""


def charger_une(con, table: str, motif: str, forme: str, regle: str,
                fenetre: Fenetre = None) -> tuple:
    """Charge les fichiers nouveaux ou modifiés d'une livraison. Rend (fichiers, lignes)."""
    nouveaux = a_charger(con, table, motif, regle, fenetre)
    if not nouveaux:
        return 0, 0

    source = LECTEURS[forme]
    noms = [f for f, _, _ in nouveaux]
    charge_le = datetime.now(timezone.utc)
    con.execute("BEGIN")
    try:
        # La table naît de la forme du premier fichier : ses colonnes, en texte, puis le lignage.
        con.execute(f"""
            CREATE TABLE IF NOT EXISTS raw.{table} AS
            SELECT * EXCLUDE (filename), filename AS _fichier, '' AS _lot, NOW() AS _charge_le
            FROM ({source}) LIMIT 0""", {"fichiers": noms[:1]})
        con.execute("CREATE OR REPLACE TEMP TABLE a_charger "
                    "(fichier VARCHAR, empreinte VARCHAR, lot VARCHAR)")
        con.executemany("INSERT INTO a_charger VALUES (?, ?, ?)", nouveaux)
```



© 2026 iAlexMG. Tous droits réservés.

```
            # Un fichier déjà chargé sous une autre empreinte : ses anciennes lignes partent d'abord.
            con.execute(f"DELETE FROM raw.{table} WHERE _fichier IN (SELECT fichier FROM a_charger)")
            con.execute(f"""
                INSERT INTO raw.{table}
                SELECT s.* EXCLUDE (filename), s.filename, a.lot, $charge_le
                FROM ({source}) s JOIN a_charger a ON a.fichier = s.filename""",
                        {"fichiers": noms, "charge_le": charge_le})
            con.execute(f"""
                INSERT INTO pipeline.chargements
                SELECT a.fichier, a.empreinte, '{table}', a.lot, COUNT(r._fichier), $charge_le
                FROM a_charger a LEFT JOIN raw.{table} r
                  ON r._fichier = a.fichier AND r._charge_le = $charge_le
                GROUP BY ALL""", {"charge_le": charge_le})
            lignes = con.execute(f"SELECT COUNT(*) FROM raw.{table} WHERE _charge_le = ?",
                                 [charge_le]).fetchone()[0]
            con.execute("COMMIT")
        except Exception:
            # Qui ouvre la transaction la referme. Sans ce ROLLBACK, DuckDB la garde avortée, et la
            # requête suivante, y compris celle qui ferme la ligne de l'étape, répond « Current
            # transaction is aborted » au lieu de laisser passer la vraie cause.
            con.execute("ROLLBACK")
            raise
        LOG.debug("%s : %d fichier(s), %d lignes", table, len(noms), lignes)
        return len(noms), lignes


def sources_de(livraisons=None) -> list:
    """Les livraisons demandées, par leur préfixe : « erp », « telemetrie »… Toutes par défaut.

    Chaque DAG du Cours 06 nomme ainsi ce qu'il possède : le mensuel prend l'ERP et les garanties,
    le quotidien prend l'usine, la télémétrie, la météo et le corpus. Deux DAG qui se partagent les
    livraisons ne se marchent pas dessus, et chacun se lit sans deviner ce que l'autre fait.
    """
    if not livraisons:
        return LIVRAISONS
    connus = sorted({table.split("_")[0] for table, _, _, _ in LIVRAISONS})
    inconnus = [nom for nom in livraisons if nom not in connus]
    if inconnus:
        # Une erreur du domaine, jamais un SystemExit : celui-ci n'est pas une Exception, il
        # traverserait le gestionnaire d'étape sans fermer sa ligne ni passer par le journal.
        raise LivraisonInconnue(f"Livraison inconnue : {', '.join(inconnus)}. "
                                f"Connues : {', '.join(connus)}.")
```



© 2026 iAlexMG. Tous droits réservés.

```
    return [ligne for ligne in LIVRAISONS if ligne[0].split("_")[0] in livraisons]


def executer(con, fenetre: Fenetre = None, livraisons=None) -> int:
    """Charge les livraisons demandées. Rend le nombre de lignes chargées à ce passage.

    Comme toute étape du pipeline, elle rend un entier : c'est ce que la porte d'entrée inscrit dans
    la colonne lignes de pipeline.executions. Le compte des fichiers, lui, va au journal de texte.

    Sans fenêtre ni liste de livraisons, elle se comporte exactement comme au Cours 05 : les 22
    livraisons, et tout ce que le journal ignore.
    """
    fenetre = fenetre or Fenetre()
    total_fichiers = total_lignes = 0
    for table, motif, forme, regle in sources_de(livraisons):
        fichiers, lignes = charger_une(con, table, motif, forme, regle, fenetre)
        if fichiers:
            LOG.info("  %-26s %4d fichier(s), %8s lignes", table, fichiers,
                     f"{lignes:,}".replace(",", " "))
        else:
            LOG.info("  %-26s    à jour", table)
        total_fichiers += fichiers
        total_lignes += lignes
    LOG.info("  RAW : %s fichier(s) chargé(s) à ce passage", f"{total_fichiers:,}".replace(",", " "))
    return total_lignes
```

3.2 Le module du bronze, qui reçoit la fenêtre et n'en fait rien. Fenêtre 1 :

```
notepad 01_de\pipeline\bronze.py
```

```
# -*- coding: utf-8 -*-
"""bronze.py : les scripts SQL du Cours 02, lancés par le pipeline."""
from pathlib import Path

from .config import DEPOT, chemin
from .journal import LOG

# Le script, et le dossier parent que son COPY partitionné attend.
SCRIPTS = [
    ("historien", "bronze_historien.sql", "bronze/usine"),
    ("télémétrie", "bronze_telemetrie.sql", "bronze/usine"),
]
```



© 2026 iAlexMG. Tous droits réservés.

```python
class EcartDeConversion(RuntimeError):
    """La source et le bronze ne comptent pas le même nombre de lignes."""


def espacer(nombre: int) -> str:
    """275436 devient « 275 436 », comme partout ailleurs dans la formation."""
    return f"{nombre:,}".replace(",", " ")


def convertir(con, script: Path) -> tuple:
    """Exécute un script de conversion et rend son contrôle final : (lignes source, lignes bronze).

    Le script porte sa requête de contrôle en dernière instruction : c'est son résultat qu'on lit.
    """
    resultat = con.execute(script.read_text(encoding="utf-8")).fetchone()
    if resultat is None or len(resultat) != 2:
        raise EcartDeConversion(f"{script.name} ne finit pas par un contrôle à deux colonnes.")
    return int(resultat[0]), int(resultat[1])


def executer(con, fenetre=None, livraisons=None) -> int:
    """Refait le bronze de l'historien et de la télémétrie. Rend le total des lignes écrites.

    La fenêtre ne s'applique pas ici : le COPY du Cours 02 réécrit ses partitions avec OVERWRITE, et
    son contrôle compare les totaux des deux côtés. Reconstruire tout coûte quelques secondes et
    garde la garantie ; ne réécrire qu'un mois demanderait un contrôle par partition.
    """
    total = 0
    for nom, fichier, parent in SCRIPTS:
        (chemin("lac") / parent).mkdir(parents=True, exist_ok=True)
        source, bronze = convertir(con, DEPOT / "sql" / fichier)
        if source != bronze:
            # Les nombres se mettent en forme un par un : appliquer le remplacement à la phrase
            # entière emporterait aussi sa virgule de ponctuation.
            raise EcartDeConversion(
                f"{nom} : {espacer(source)} lignes dans la source, {espacer(bronze)} dans le "
                "bronze. La conversion a perdu ou ajouté des lignes.")
        LOG.info("  %-12s %10s lignes, source et bronze d'accord", nom, espacer(bronze))
        total += bronze
    return total
```



© 2026 iAlexMG. Tous droits réservés.

3.3 Le module du corpus, même chose. Fenêtre 1 :

```
notepad 01_de\pipeline\corpus.py
```

```
# -*- coding: utf-8 -*-
"""corpus.py : la documentation de Boréal déposée telle quelle dans le bronze (Cours 03, empaqueté au 05).

Ce qu'il fait. Il copie chaque fichier de sources/corpus/ dans plateforme/lac/bronze/corpus/, en
gardant l'arborescence, et saute ceux dont l'empreinte n'a pas bougé. Le catalogue, lui, entre dans
la RAW comme une table, par l'étape raw.

La règle qu'il illustre. Tout ne se met pas en table. Un document se dépose tel quel, avec sa date et
son empreinte, et c'est ce dépôt que la formation AI ira lire. Déposer reste une étape du pipeline :
elle se journalise, se relance sans dégât, et rend un compte.
"""
import hashlib
import shutil

from .config import chemin
from .journal import LOG


def empreinte(fichier) -> str:
    return hashlib.sha256(fichier.read_bytes()).hexdigest()


def executer(con=None, fenetre=None, livraisons=None) -> int:
    """Dépose le corpus dans le bronze. Rend le nombre de fichiers déposés à ce passage.

    La fenêtre du Cours 06 ne s'applique pas ici : un document n'a pas de lot, et c'est son empreinte
    qui dit s'il a bougé.
    """
    source = chemin("sources") / "corpus"
    bronze = chemin("lac") / "bronze" / "corpus"
    deposes = identiques = 0
    for fichier in sorted(p for p in source.rglob("*") if p.is_file()):
        cible = bronze / fichier.relative_to(source)
        if cible.exists() and empreinte(cible) == empreinte(fichier):
            identiques += 1
            continue
        cible.parent.mkdir(parents=True, exist_ok=True)
        # copyfile, et non copy2 : recopier les métadonnées échoue quand un conteneur Linux écrit
        # sur un montage de fichiers Windows, « [Errno 1] Operation not permitted », et l'étape n'en
```



© 2026 iAlexMG. Tous droits réservés.

```
            # a pas besoin. C'est l'empreinte du contenu qui dit si un document a bougé, pas sa date.
            # Trouvé au Cours 06, la première fois qu'Airflow a lancé cette étape depuis un conteneur.
            shutil.copyfile(fichier, cible)
            deposes += 1
        LOG.info("  corpus : %d fichier(s) déposé(s), %d déjà à jour", deposes, identiques)
        return deposes
```

3.4 Le module de la base de l'ERP, même chose. Fenêtre 1 :

```
notepad 01_de\pipeline\erp.py
```

```
# -*- coding: utf-8 -*-
"""erp.py : les neuf tables de la base de l'ERP, lues par ODBC (Cours 04, empaqueté au Cours 05).

Ce qu'il fait. Il se connecte à la base de l'ERP de Boréal, dans son conteneur PostgreSQL, par le
pilote psqlODBC, lit chaque table du schéma sap par paquets, l'écrit dans un CSV temporaire, puis
DuckDB la range en Parquet dans plateforme/lac/bronze/erp_base/.

La règle qu'il illustre. Une dépendance qui ne sert qu'à une étape s'importe dans cette étape, pas en
tête du paquet. pyodbc et le pilote psqlODBC ne s'installent que pour cette extraction : le reste du
pipeline, celui que rejoue le kit 02, n'importe que DuckDB et la bibliothèque standard, et tourne
même quand pyodbc n'est pas là. Le mot de passe, lui, n'est ni dans le code ni dans config.toml : il
est lu par config.secret, dans 01_de/.env ou dans l'environnement du conteneur.

Lancement : python -m pipeline erp, le conteneur de la base démarré.
"""
import csv
import time

import duckdb

from .config import config, chemin, secret
from .journal import LOG


class PiloteAbsent(RuntimeError):
    """Aucun pilote ODBC de PostgreSQL n'est installé sur ce poste."""


def chaine_de_connexion(pyodbc) -> str:
    """La chaîne ODBC, pilote compris. Changer de base ne change que cette chaîne."""
    pilotes = [p for p in pyodbc.drivers() if p.startswith("PostgreSQL Unicode")]
    if not pilotes:
```



© 2026 iAlexMG. Tous droits réservés.

```
        raise PiloteAbsent(f"Pilote psqlODBC introuvable. Pilotes ODBC installés : {pyodbc.drivers()}")
    reglages = config()["erp"]
    return (f"DRIVER={{{pilotes[0]}}};SERVER={reglages['serveur']};PORT={reglages['port']};"
            f"DATABASE={reglages['base']};UID={reglages['utilisateur']};"
            f"PWD={secret('ERP_MOT_DE_PASSE')}")


def executer(con=None, fenetre=None, livraisons=None) -> int:
    """Extrait les neuf tables de la base. Rend le total des lignes lues.

    La fenêtre du Cours 06 ne s'applique pas ici : l'ERP livre une base, pas des fichiers datés, et
    c'est justement ce que le Cours 04 oppose aux extraits mensuels.
    """
    import pyodbc                                   # dépendance de cette étape seulement

    reglages = config()["erp"]
    sortie = chemin("lac") / "bronze" / "erp_base"
    sortie.mkdir(parents=True, exist_ok=True)

    debut = time.perf_counter()
    total = 0
    connexion = pyodbc.connect(chaine_de_connexion(pyodbc))
    curseur = connexion.cursor()
    duck = duckdb.connect()                         # une base en mémoire : l'entrepôt n'est pas touché
    for table in reglages["tables"]:
        curseur.execute(f"SELECT * FROM sap.{table}")
        colonnes = [c[0].upper() for c in curseur.description]
        temporaire = sortie / f"{table}.csv"
        lignes = 0
        with open(temporaire, "w", newline="", encoding="utf-8") as fichier:
            ecrivain = csv.writer(fichier)
            ecrivain.writerow(colonnes)
            while paquet := curseur.fetchmany(reglages["paquet"]):
                ecrivain.writerows(paquet)
                lignes += len(paquet)
        # Le bronze garde le texte reçu ; DuckDB le range en Parquet compressé.
        duck.execute(f"""
            COPY (SELECT * FROM read_csv('{temporaire.as_posix()}', header = true, all_varchar = true))
            TO '{(sortie / f"{table}.parquet").as_posix()}' (FORMAT parquet, COMPRESSION zstd)""")
        temporaire.unlink()
        LOG.info("  sap.%-8s %8s lignes", table, f"{lignes:,}".replace(",", " "))
        total += lignes
```



© 2026 iAlexMG. Tous droits réservés.

```
    connexion.close()
    LOG.info("  neuf tables extraites en %.1f s", time.perf_counter() - debut)
    return total
```

3.5 Le module de l'API météo, même chose. Fenêtre 1 :

```
notepad 01_de\pipeline\meteo.py
```

```
# -*- coding: utf-8 -*-
"""meteo.py : la vraie API d'ECCC, page par page, avec les reprises du pipeline (Cours 05).

Ce qu'il fait. Il demande à api.weather.gc.ca les observations quotidiennes de la station configurée,
page par page, en suivant le lien « next » que chaque réponse fournit, et écrit chaque observation
telle que reçue, une ligne JSON par jour, dans plateforme/lac/bronze/meteo_api/. À la fin, il compare
les jours reçus aux jours annoncés par l'API et refuse une extraction incomplète.

Ce que le Cours 05 a changé, par rapport au script du Cours 04. La boucle d'essais du script est
partie dans pipeline.reprise, où elle sert à toute source qui peut faiblir : le module ne garde que
ce qui lui est propre, savoir quelles erreurs HTTP valent un second essai. La station, les dates et
la taille de page viennent de la configuration, et les messages vont au journal.

La règle qu'il illustre. Ce qui se répète dans deux étapes n'appartient à aucune des deux. Une
reprise, une journalisation, une lecture de configuration sont l'affaire du pipeline ; l'étape ne
garde que la connaissance de sa source.
"""
import json
import urllib.error
import urllib.parse
import urllib.request

from .config import config, chemin
from .journal import LOG
from .reprise import avec_reprises


class ExtractionIncomplete(RuntimeError):
    """L'API a annoncé plus de jours qu'elle n'en a livré."""


def passagere(erreur: Exception) -> bool:
    """Un 429 ou un 5xx passeront peut-être ; un 4xx dit que la demande est fausse."""
    if isinstance(erreur, urllib.error.HTTPError):
        return erreur.code == 429 or erreur.code >= 500
```



© 2026 iAlexMG. Tous droits réservés.

```
    return isinstance(erreur, urllib.error.URLError)     # coupure, DNS, délai dépassé


def lire_page(url: str) -> dict:
    """Une page de l'API, réessayée tant que l'erreur est passagère."""
    def demander():
        with urllib.request.urlopen(url, timeout=60) as reponse:
            return json.load(reponse)
    return avec_reprises(demander, passagere, "une page de l'API météo")


def executer(con=None, fenetre=None, livraisons=None) -> int:
    """Tire les observations de la station configurée. Rend le nombre de jours reçus.

    La fenêtre du Cours 06 ne s'applique pas ici : l'API porte ses propres dates, dans config.toml.
    Les brancher sur la fenêtre est la question pratique 6 du Cours 06.
    """
    reglages = config()["meteo"]
    parametres = {"f": "json", "CLIMATE_IDENTIFIER": reglages["station"], "sortby": "LOCAL_DATE",
                  "datetime": f"{reglages['debut']} 00:00:00/{reglages['fin']} 00:00:00",
                  "limit": reglages["page"]}
    url = f"{reglages['url']}?{urllib.parse.urlencode(parametres)}"
    sortie = (chemin("lac") / "bronze" / "meteo_api" /
              f"quotidien_{reglages['station']}_{reglages['debut']}_{reglages['fin']}.jsonl")
    sortie.parent.mkdir(parents=True, exist_ok=True)

    pages = recus = 0
    annonces = None
    with open(sortie, "w", encoding="utf-8", newline="\n") as fichier:
        while url:
            page = lire_page(url)
            pages += 1
            annonces = page["numberMatched"]
            for entite in page["features"]:
                fichier.write(json.dumps(entite["properties"], ensure_ascii=False) + "\n")
            recus += page["numberReturned"]
            LOG.debug("  page %d : %d jours", pages, page["numberReturned"])
            # La page suivante, si l'API en annonce une.
            url = next((lien["href"] for lien in page["links"] if lien["rel"] == "next"), None)

    LOG.info("  météo : %d jours reçus sur %d annoncés, en %d pages", recus, annonces, pages)
    if recus != annonces:
```



© 2026 iAlexMG. Tous droits réservés.

```
        raise ExtractionIncomplete(
            f"{recus} jours reçus pour {annonces} annoncés : l'extraction est incomplète.")
    return recus
```

3.6 La porte d'entrée, qui lit `--depuis`, `--jusqua` et `--livraisons`. Fenêtre 1 :

```
notepad 01_de\pipeline\__main__.py
```

```
# -*- coding: utf-8 -*-
"""__main__.py : la porte d'entrée du pipeline de Boréal (Cours 05).

Ce qu'il fait. Il lit la ligne de commande, ouvre le journal et l'entrepôt une seule fois, puis lance
les étapes demandées, chacune dans son encadrement : une ligne dans pipeline.executions au début, sa
durée et son compte à la fin, et l'erreur inscrite si elle échoue.

    python -m pipeline bronze raw corpus     les étapes nommées, dans cet ordre
    python -m pipeline tout                  les étapes du chemin hors ligne, dans le bon ordre
    python -m pipeline raw --bavard          le journal descend au niveau DEBUG
    python -m pipeline --etapes              la liste des étapes, sans rien lancer

    python -m pipeline raw --depuis 2026-01 --jusqua 2026-01        un seul mois (Cours 06)
    python -m pipeline raw --depuis 2026-01-14 --jusqua 2026-01-14  une seule journée
    python -m pipeline raw --livraisons erp,garanties                seulement ces livraisons

La règle qu'il illustre. Un pipeline a une seule porte d'entrée. Le DAG du Cours 06 appellera cette
commande, tâche par tâche ; le kit de départ d'Analytics Engineering la rejouera sans Airflow ; le
conteneur la lance comme point d'entrée de l'image. Trois façons d'appeler, un seul chemin de code.

Le code de sortie compte autant que le journal : 0 quand tout est passé, 1 quand une étape a échoué.
C'est ce qu'Airflow lit, et c'est pour lui que main() ne laisse jamais filer une exception.

Deux étapes ne sont pas dans « tout » : erp, qui demande le conteneur de la base et le pilote
psqlODBC, et meteo, qui demande le réseau. Un pipeline rejouable hors ligne ne les suppose pas.
"""
import argparse
import os
import sys

from . import bronze, corpus, erp, meteo, raw
from .config import projet
from .entrepot import ouvrir
from .fenetre import DateIllisible, Fenetre
from .journal import LOG, demarrer, etape
```



© 2026 iAlexMG. Tous droits réservés.

```python
# Le nom de l'étape, ce qu'elle fait, si « tout » la prend, et si une fenêtre la restreint.
# La dernière colonne est celle du Cours 06 : seule l'étape raw travaille livraison par livraison,
# donc elle seule peut se limiter à une tranche de temps. Les autres reconstruisent tout, par
# conception : le bronze réécrit ses partitions (OVERWRITE, Cours 02), le corpus compare des
# empreintes, l'ERP lit une base entière et la météo appelle une API qui porte ses propres dates.
ETAPES = {
    "bronze": (bronze.executer, "les CSV et le JSON de l'usine, rangés en Parquet", True, False),
    "raw": (raw.executer, "les 22 livraisons chargées dans la couche RAW", True, True),
    "corpus": (corpus.executer, "la documentation déposée telle quelle dans le bronze", True, False),
    "erp": (erp.executer, "les neuf tables de la base de l'ERP, par ODBC", False, False),
    "meteo": (meteo.executer, "les observations d'ECCC, par son API", False, False),
}


def lire_arguments(arguments=None) -> argparse.Namespace:
    analyseur = argparse.ArgumentParser(
        prog="python -m pipeline",
        description="Le pipeline de Véhicules Boréal inc. : une étape, plusieurs, ou tout.")
    analyseur.add_argument("etapes", nargs="*", metavar="ÉTAPE",
                           help="les étapes à lancer, ou « tout »")
    analyseur.add_argument("--bavard", action="store_true", help="journal au niveau DEBUG")
    analyseur.add_argument("--etapes", dest="lister", action="store_true",
                           help="liste les étapes et ce qu'elles font, sans rien lancer")
    analyseur.add_argument("--depuis", metavar="AAAA-MM[-JJ]",
                           help="premier lot traité ; sans lui, tout l'historique")
    analyseur.add_argument("--jusqua", metavar="AAAA-MM[-JJ]",
                           help="dernier lot traité ; sans lui, jusqu'au dernier livré")
    analyseur.add_argument("--livraisons", metavar="erp,garanties",
                           help="les livraisons traitées, par préfixe ; sans elles, les 22")
    return analyseur.parse_args(arguments)


def resoudre(demandees: list) -> list:
    """La liste des étapes à lancer, dans l'ordre. « tout » prend celles du chemin hors ligne."""
    if demandees == ["tout"]:
        return [nom for nom, (_, _, dans_tout, _) in ETAPES.items() if dans_tout]
    inconnues = [nom for nom in demandees if nom not in ETAPES]
    if inconnues:
        raise SystemExit(f"Étape inconnue : {', '.join(inconnues)}. "
                         f"Connues : {', '.join(ETAPES)}, ou « tout ».")
    return demandees
```



© 2026 iAlexMG. Tous droits réservés.

```
def main(arguments=None) -> int:
    options = lire_arguments(arguments)
    if options.lister or not options.etapes:
        print("Étapes du pipeline :")
        for nom, (_, quoi, dans_tout, fenetree) in ETAPES.items():
            marques = ([] if dans_tout else ["hors de « tout »"]) + \
                      (["fenêtre"] if fenetree else ["reconstruit tout"])
            print(f"  {nom:8} {quoi}   ({', '.join(marques)})")
        return 0

    try:
        fenetre = Fenetre.depuis_la_ligne_de_commande(options.depuis, options.jusqua)
    except DateIllisible as erreur:
        raise SystemExit(str(erreur))
    if fenetre.depuis and fenetre.jusqua and fenetre.depuis > fenetre.jusqua:
        raise SystemExit(f"Fenêtre vide : --depuis {options.depuis} vient après "
                         f"--jusqua {options.jusqua}.")

    livraisons = [nom.strip() for nom in options.livraisons.split(",")] if options.livraisons else None

    a_lancer = resoudre(options.etapes)
    # Le pipeline travaille depuis Projet_Boreal_intermediaire, d'où qu'on l'appelle : les chemins des scripts SQL
    # et le lignage de la RAW s'écrivent alors toujours de la même façon.
    os.chdir(projet())
    if options.bavard:
        os.environ["PIPELINE_JOURNAL_NIVEAU"] = "DEBUG"
    demarrer()
    LOG.info("pipeline : %s, depuis %s", ", ".join(a_lancer), projet())
    LOG.info("fenêtre : %s", fenetre)
    LOG.info("livraisons : %s", ", ".join(livraisons) if livraisons else "toutes")
    if not fenetre.ouverte:
        ignorees = [nom for nom in a_lancer if not ETAPES[nom][3]]
        if ignorees:
            # Un avertissement, pas une erreur : un DAG quotidien lance bronze et raw dans la même
            # commande, et il est normal que seule la seconde se restreigne.
            LOG.warning("fenêtre ignorée par %s : cette étape reconstruit tout",
                        ", ".join(ignorees))

    con = ouvrir()
    try:
```



© 2026 iAlexMG. Tous droits réservés.

```
            for nom in a_lancer:
                action = ETAPES[nom][0]
                with etape(con, nom) as trace:
                    # Une étape rend un entier, ce qu'elle a écrit. Le convertir ici fait dire à une
                    # étape mal écrite ce qu'elle a fait de travers, plutôt qu'à DuckDB deux lignes
                    # plus loin, dans une erreur de conversion de type.
                    resultat = action(con, fenetre, livraisons)
                    trace.lignes = None if resultat is None else int(resultat)
        except Exception as erreur:
            LOG.error("pipeline arrêté : %s", erreur)
            return 1
        finally:
            con.close()
        LOG.info("pipeline : terminé")
        return 0


    if __name__ == "__main__":
        sys.stdout.reconfigure(encoding="utf-8", errors="replace")
        sys.exit(main())
```

3.7 Voir les étapes, et celles qu'une fenêtre restreint. Fenêtre 1 :

```
python -m pipeline --etapes
REM Attendu : cinq lignes ; raw porte (fenêtre), les quatre autres (reconstruit tout),
REM et erp et meteo, en plus, (hors de « tout »).
```

## Étape 4 — Charger une seule journée, et voir les garde-fous répondre

*Théorie : `Theorie_06_DE.md`, annexe A.4.*

4.1 Charger la télémétrie du 14 janvier 2026, et elle seule. Fenêtre 1 :

```
python -m pipeline raw --livraisons telemetrie --depuis 2026-01-14 --jusqua 2026-01-14
REM Attendu : fenêtre : du 2026-01-14 au 2026-01-14, livraisons : telemetrie,
REM telemetrie_sorties 1 fichier(s), 222 lignes, puis RAW : 1 fichier(s) chargé(s) à ce passage.
```

4.2 Relancer la même commande, pour prouver que le journal ne recharge rien. Fenêtre 1 :

```
python -m pipeline raw --livraisons telemetrie --depuis 2026-01-14 --jusqua 2026-01-14
REM Attendu : telemetrie_sorties à jour, puis RAW : 0 fichier(s) chargé(s) à ce passage.
```

4.3 Erreur voulue : un « s » oublié dans le nom d'une livraison. Fenêtre 1 :



© 2026 iAlexMG. Tous droits réservés.

```
python -m pipeline raw --livraisons erp,garantie
REM Attendu : une erreur, Livraison inconnue : garantie. Connues : corpus, erp, garanties, meteo, telemetrie, usine.
```

4.4 Erreur voulue : un mois qui n'existe pas. Fenêtre 1 :

```
python -m pipeline raw --depuis 2026-13
REM Attendu : une erreur, « 2026-13 » n'est pas une date : month must be in 1..12
```

4.5 Erreur voulue : deux bornes inversées. Fenêtre 1 :

```
python -m pipeline raw --depuis 2026-03 --jusqua 2026-01
REM Attendu : une erreur, Fenêtre vide : --depuis 2026-03 vient après --jusqua 2026-01.
```

À retenir — un garde-fou qui parle vaut mieux qu'un filtre muet. Sans ces trois arrêts, un « s » oublié ou deux bornes inversées donneraient une exécution verte qui n'aurait rien chargé : le pire résultat, faux et silencieux. On les provoque ici pour les reconnaître le jour où un DAG les déclenchera.

## Étape 5 — Tester le rattrapage sans Airflow

*Théorie : `Theorie_06_DE.md`, annexe A.5.*

5.1 Ajouter deux tests, qui vérifient la fenêtre et le rattrapage. Le fichier existe déjà : tout sélectionner, coller cette version à sa place, enregistrer, fermer. Fenêtre 1 :

```
notepad 01_de\tests\test_pipeline.py
```

```
# -*- coding: utf-8 -*-
"""test_pipeline.py : trois propriétés du pipeline, vérifiées sans les données de Boréal."""
import urllib.error

import pytest

from pipeline import bronze, raw
from pipeline.config import config
from pipeline.entrepot import ouvrir
from pipeline.fenetre import DateIllisible, Fenetre
from pipeline.meteo import passagere
from pipeline.reprise import Abandon, avec_reprises


@pytest.fixture
def projet(tmp_path, monkeypatch):
    """Un Projet_Boreal_intermediaire jetable : le pipeline y écrit son entrepôt et ses fichiers."""
    monkeypatch.setenv("PIPELINE_PROJET", str(tmp_path))
```



© 2026 iAlexMG. Tous droits réservés.

```
    config.cache_clear()
    monkeypatch.chdir(tmp_path)
    yield tmp_path
    config.cache_clear()


def ecrire_extrait(projet, mois: str, lignes: list) -> None:
    dossier = projet / "sources" / "erp" / "extraits" / mois
    dossier.mkdir(parents=True, exist_ok=True)
    contenu = "VBELN,NETWR\n" + "".join(f"{v}\n" for v in lignes)
    (dossier / "VBAK.csv").write_text(contenu, encoding="utf-8")


def test_le_journal_refuse_un_extrait_deja_charge(projet):
    """Recharger le même fichier ne double rien ; le même fichier corrigé remplace ses lignes."""
    ecrire_extrait(projet, "2023-01", ["0000000001,100", "0000000002,200"])
    con = ouvrir()
    try:
        motif, table = "sources/erp/extraits/*/VBAK.csv", "erp_vbak"

        fichiers, lignes = raw.charger_une(con, table, motif, "csv", "dossier")
        assert (fichiers, lignes) == (1, 2)

        # Deuxième passage, rien n'a changé : le journal reconnaît l'empreinte et ne recharge rien.
        assert raw.charger_une(con, table, motif, "csv", "dossier") == (0, 0)
        assert con.execute(f"SELECT COUNT(*) FROM raw.{table}").fetchone()[0] == 2

        # Le même nom, un autre contenu : ses anciennes lignes partent, les nouvelles entrent.
        ecrire_extrait(projet, "2023-01", ["0000000001,100", "0000000002,200", "0000000003,300"])
        assert raw.charger_une(con, table, motif, "csv", "dossier") == (1, 3)
        assert con.execute(f"SELECT COUNT(*) FROM raw.{table}").fetchone()[0] == 3

        # Un autre nom de dossier est une autre livraison : le journal la laisse passer, et la RAW
        # garde les deux lots. C'est le piège de 2023-03_bis, et il est voulu.
        ecrire_extrait(projet, "2023-01_bis", ["0000000001,100"])
        assert raw.charger_une(con, table, motif, "csv", "dossier") == (1, 1)
        assert con.execute(f"SELECT COUNT(DISTINCT _lot) FROM raw.{table}").fetchone()[0] == 2
    finally:
        con.close()


def test_la_conversion_garde_le_compte_des_lignes(projet, tmp_path):
```



© 2026 iAlexMG. Tous droits réservés.

```
    """Le contrôle d'un script de conversion rend le même compte des deux côtés, ou il lève."""
    source = projet / "mesures"
    source.mkdir()
    for jour, valeurs in (("2025-10-01", [1.0, 2.0]), ("2025-10-02", [3.0])):
        (source / f"mesures_{jour}.csv").write_text(
            "Tag,Value\n" + "".join(f"UTL.CMP2.VIBRATION,{v}\n" for v in valeurs), encoding="utf-8")

    script = tmp_path / "conversion.sql"
    script.write_text("""
        COPY (
            SELECT Tag, Value,
                   CAST(REPLACE(PARSE_FILENAME(filename, true), 'mesures_', '') AS DATE) AS jour
            FROM read_csv('mesures/*.csv', header = true, filename = true,
                          columns = {'Tag': 'VARCHAR', 'Value': 'DOUBLE'})
        ) TO 'bronze/mesures' (FORMAT parquet, PARTITION_BY (jour), OVERWRITE);
        SELECT (SELECT COUNT(*) FROM 'mesures/*.csv') AS lignes_csv,
               (SELECT COUNT(*) FROM 'bronze/mesures/*/*.parquet') AS lignes_parquet;
        """, encoding="utf-8")
    (projet / "bronze").mkdir()

    con = ouvrir()
    try:
        assert bronze.convertir(con, script) == (3, 3)

        # Un script qui ne finit pas par un contrôle à deux colonnes est refusé, plutôt que cru.
        muet = tmp_path / "muet.sql"
        muet.write_text("SELECT 1;", encoding="utf-8")
        with pytest.raises(bronze.EcartDeConversion):
            bronze.convertir(con, muet)
    finally:
        con.close()


def test_la_reprise_s_arrete_apres_le_nombre_d_essais_prevu(projet):
    """Quatre essais, trois attentes qui doublent, puis l'abandon. Une erreur définitive ne passe pas."""
    essais = []
    attentes = []

    def toujours_en_panne():
        essais.append(1)
        raise urllib.error.HTTPError("https://exemple", 503, "Service Unavailable", None, None)
```



© 2026 iAlexMG. Tous droits réservés.

```
    with pytest.raises(Abandon):
        avec_reprises(toujours_en_panne, passagere, "essai", essais=4, attente=2.0,
                      dormir=attentes.append)
    assert len(essais) == 4
    assert attentes == [2.0, 4.0, 8.0]

    # Un 404 dit que la demande est fausse : une seule tentative, et l'erreur remonte telle quelle.
    definitive = []

    def demande_fausse():
        definitive.append(1)
        raise urllib.error.HTTPError("https://exemple", 404, "Not Found", None, None)

    with pytest.raises(urllib.error.HTTPError):
        avec_reprises(demande_fausse, passagere, "essai", essais=4, attente=2.0,
                      dormir=attentes.append)
    assert len(definitive) == 1


def test_la_fenetre_ne_retient_que_les_lots_de_sa_tranche():
    """Un mois, un jour, une relivraison, un export complet : chacun à sa place, sans base ni fichier."""
    janvier = Fenetre.depuis_la_ligne_de_commande("2026-01", "2026-01")
    assert janvier.contient("2026-01")
    assert not janvier.contient("2025-12")
    assert not janvier.contient("2026-02")

    # Un lot du jour : la fenêtre du mois le prend, celle d'une autre journée non.
    assert janvier.contient("2026-01-14")
    assert not Fenetre.depuis_la_ligne_de_commande("2026-01-14", "2026-01-14").contient("2026-01-15")

    # Une relivraison garde la date de son mois : 2023-03_bis est bien du mois de mars 2023.
    mars = Fenetre.depuis_la_ligne_de_commande("2023-03", "2023-03")
    assert mars.contient("2023-03_bis")

    # Ce qui n'a pas de date entre partout : c'est le journal qui décide ensuite.
    assert janvier.contient("complet")
    assert janvier.contient("un_lot_inattendu")

    # Sans borne, la fenêtre est ouverte : le pipeline se comporte comme au Cours 05.
    assert Fenetre().ouverte and Fenetre().contient("2019-07")

    # Une borne illisible se dit tout de suite, plutôt que de filtrer de travers.
```



© 2026 iAlexMG. Tous droits réservés.

```
    with pytest.raises(DateIllisible):
        Fenetre.depuis_la_ligne_de_commande("janvier 2026")


def test_trois_fenetres_chargent_ce_qu_une_seule_aurait_charge(projet):
    """Le rattrapage mois par mois donne la même RAW qu'un chargement d'un bloc. C'est la promesse
    que le DAG mensuel tient, et elle vaut d'être vérifiée sans Airflow."""
    for mois, lignes in (("2026-01", ["0000000001,100", "0000000002,200"]),
                         ("2026-02", ["0000000003,300"]),
                         ("2026-03", ["0000000004,400", "0000000005,500"])):
        ecrire_extrait(projet, mois, lignes)
    motif, table = "sources/erp/extraits/*/VBAK.csv", "erp_vbak"

    con = ouvrir()
    try:
        # Le rattrapage : une exécution par mois, chacune dans sa tranche.
        for mois in ("2026-01", "2026-02", "2026-03"):
            fenetre = Fenetre.depuis_la_ligne_de_commande(mois, mois)
            fichiers, _ = raw.charger_une(con, table, motif, "csv", "dossier", fenetre)
            assert fichiers == 1, f"{mois} aurait dû charger son seul fichier"

        par_lot = dict(con.execute(
            f"SELECT _lot, COUNT(*) FROM raw.{table} GROUP BY _lot").fetchall())
        assert par_lot == {"2026-01": 2, "2026-02": 1, "2026-03": 2}

        # Une exécution de plus, fenêtre ouverte : le journal a tout vu, il ne reste rien.
        assert raw.charger_une(con, table, motif, "csv", "dossier") == (0, 0)

        # Et deux fenêtres qui se chevauchent ne doublent rien : c'est le journal qui garde
        # l'idempotence, pas la fenêtre.
        large = Fenetre.depuis_la_ligne_de_commande("2026-01", "2026-03")
        assert raw.charger_une(con, table, motif, "csv", "dossier", large) == (0, 0)
        assert con.execute(f"SELECT COUNT(*) FROM raw.{table}").fetchone()[0] == 5
    finally:
        con.close()
```

5.2 Lancer les tests. Fenêtre 1 :

```
pytest 01_de
REM Attendu : collected 5 items, puis 5 passed, en moins d'une seconde (0,4 s chez l'auteur).
```

Point d’arrêt. Le pipeline sait traiter une tranche de temps, et il le prouve sans Airflow : c’est la moitié du cours, et elle se tient seule. Qui fait ce labo en deux séances s’arrête ici. À la reprise, refaire « Avant de commencer », construction de la pile comprise si elle n’est pas faite.



© 2026 iAlexMG. Tous droits réservés.

## Étape 6 — Allumer Airflow et lire les fichiers qui le composent

*Théorie : `Theorie_06_DE.md`, annexe A.6.*

6.1 Vérifier que la pile tourne. La commande de démarrage ne fait rien si tout tourne déjà, et rallume ce qui s'est arrêté. Fenêtre 2 :

```
docker compose up -d airflow-apiserver airflow-scheduler airflow-dag-processor airflow-triggerer
# Attendu : Running pour chaque service, ou Started pour ceux qui s'étaient arrêtés.
docker compose ps
# Attendu : les cinq services d'Airflow (healthy), et erp.
```

6.2 Ouvrir l'interface d'Airflow, dans le navigateur, à `http://localhost:8080`, avec le compte `airflow` et le mot de passe `airflow`. Attendu : la liste des DAG, vide.

6.3 Créer le dossier où Airflow cherche les DAG, dans votre dépôt. Fenêtre 1 :

```
mkdir 01_de\dags
REM Attendu : rien. Airflow le trouvera seul, sans redémarrage.
```

6.4 Lire la recette de l'image d'Airflow, reçue avec la plateforme. Fenêtre 1 :

```
type plateforme\Dockerfile.airflow
REM Attendu : FROM apache/airflow:3.3.1, puis l'ajout du pilote ODBC et de DuckDB 1.5.5.
```

6.5 Lire la description de la pile, sans la modifier : la plateforme appartient à l'équipe qui l'exploite. Fenêtre 1 :

```
notepad plateforme\docker-compose.yml
```

Lire, puis fermer sans enregistrer. L'annexe A.6 dit ce que porte chaque morceau.

## Étape 7 — Vérifier le pool de l'entrepôt

*Théorie : `Theorie_06_DE.md`, annexe A.7.*

7.1 Lister les pools d'Airflow. La plateforme a posé le verrou de l'entrepôt au premier démarrage. Fenêtre 2 :

```
docker compose exec airflow-scheduler airflow pools list
# Attendu : default_pool, 128 places, et entrepot, 1 place, « Un seul écrivain sur boreal.duckdb ».
```

## Étape 8 — Écrire le DAG quotidien et lancer une journée

*Théorie : `Theorie_06_DE.md`, annexe A.8.*

8.1 Écrire la fonction que l'alerte de retard appellera. Fenêtre 1 :



© 2026 iAlexMG. Tous droits réservés.

```
notepad 01_de\dags\boreal_alertes.py
```

Accepter la création. Coller, enregistrer, fermer :

```
# -*- coding: utf-8 -*-
"""boreal_alertes.py : ce qu'on fait quand une exécution dépasse son délai.

Une alerte de retard n'est pas une reprise. Personne ne relance rien : on signale, parce qu'un
chargement qui met trois heures au lieu de dix minutes finit peut-être par passer, et c'est
justement ce silence qui coûte cher. En production, cette fonction enverrait un courriel ou un
message ; dans le labo, elle écrit, et c'est la même mécanique.

Une fonction ordinaire s'enveloppe dans SyncCallback. AsyncCallback, celui que la documentation
montre en exemple, attend une coroutine : lui donner cette fonction fait échouer la lecture du DAG
sur « Provided callback ... is not awaitable », et le DAG n'apparaît pas du tout dans l'interface.
"""
import logging

journal = logging.getLogger(__name__)


def prevenir_retard(**contexte) -> None:
    """Écrit le dépassement dans le journal. Les valeurs arrivent déjà remplies par Airflow."""
    journal.warning("RETARD : %s", contexte.get("message", contexte))
```

8.2 Écrire le DAG quotidien. Fenêtre 1 :

```
notepad 01_de\dags\boreal_quotidien.py
```

Accepter la création. Coller, enregistrer, fermer :

```
# -*- coding: utf-8 -*-
"""boreal_quotidien.py : le cold path de Boréal, une fois par jour.

Quatre tâches, dans l'ordre : le bronze de l'usine, la couche RAW limitée à la journée de
l'exécution, le dépôt du corpus, puis la place que dbt prendra. Chacune appelle la porte d'entrée du
pipeline. Airflow orchestre, le pipeline travaille.

Un DAG mélange deux sortes de tâches, et il faut savoir laquelle on écrit. « raw » est incrémentale :
elle reçoit la date de son exécution et ne charge que les livraisons de cette journée.
« bronze » reconstruit tout à chaque passage, parce que le COPY du Cours 02 réécrit ses
partitions et vérifie ses totaux. Les deux sont rejouables, par des chemins différents.
"""
from __future__ import annotations
```



© 2026 iAlexMG. Tous droits réservés.

```python
from datetime import timedelta

import pendulum
from airflow.sdk import DeadlineAlert, DeadlineReference, SyncCallback, dag
from airflow.providers.standard.operators.bash import BashOperator
from airflow.providers.standard.operators.empty import EmptyOperator
from airflow.timetables.interval import CronDataIntervalTimetable

from boreal_alertes import prevenir_retard

# Le pipeline se lance depuis le projet monté ; PIPELINE_PROJET et PYTHONPATH viennent du compose.
PIPELINE = "cd /projet/01_de && python -m pipeline"
# Le pool à une place qui garde l'entrepôt. Toute tâche qui appelle le pipeline le prend, dans ce
# DAG comme dans le mensuel : DuckDB n'accepte qu'un écrivain à la fois sur son fichier, et le
# nombre d'exécutions simultanées ne protège un DAG que de lui-même.
ENTREPOT = "entrepot"


@dag(
    dag_id="boreal_quotidien",
    # L'intervalle de données, écrit en toutes lettres : depuis Airflow 3, « @daily » tout court ne
    # découpe plus de tranche, et l'exécution de minuit chargerait la journée qui commence.
    schedule=CronDataIntervalTimetable("@daily", timezone="UTC"),
    start_date=pendulum.datetime(2026, 3, 1, tz="UTC"),
    # Sans rattrapage : l'historique a été chargé d'un bloc au Cours 05, et personne ne veut
    # 1 500 exécutions quotidiennes à la première mise en route. Le DAG mensuel, lui, rattrape.
    catchup=False,
    max_active_runs=1,
    # Deux essais, cinq minutes d'attente : une écriture DuckDB refusée parce qu'un autre processus
    # tenait l'entrepôt passe au second essai, une erreur de SQL non.
    default_args={"retries": 1, "retry_delay": timedelta(minutes=5)},
    deadline=DeadlineAlert(
        reference=DeadlineReference.DAGRUN_QUEUED_AT,
        interval=timedelta(minutes=30),
        callback=SyncCallback(
            prevenir_retard,
            kwargs={"message": "boreal_quotidien du {{ ds }} n'a pas fini en 30 minutes"},
        ),
    ),
    tags=["boreal", "cold path"],
    doc_md=__doc__,
```



© 2026 iAlexMG. Tous droits réservés.

```
)
def boreal_quotidien():
    bronze = BashOperator(
        task_id="bronze",
        bash_command=f"{PIPELINE} bronze",
        pool=ENTREPOT,
    )

    # La seule tâche fenêtrée du DAG. {{ ds }} est la date logique de l'exécution, le début de sa
    # tranche, et non le jour où elle tourne : rejouer le 15 mars en juin recharge le 15 mars. C'est
    # aussi la bonne date pour une exécution lancée à la main. Les livraisons nommées sont celles
    # que ce DAG possède ; l'ERP et les garanties arrivent par mois.
    raw = BashOperator(
        task_id="raw",
        bash_command=f"{PIPELINE} raw --livraisons usine,telemetrie,meteo,corpus "
                     f"--depuis {{{{ ds }}}} --jusqua {{{{ ds }}}}",
        pool=ENTREPOT,
    )

    corpus = BashOperator(
        task_id="corpus",
        bash_command=f"{PIPELINE} corpus",
        pool=ENTREPOT,
    )

    # La place de dbt, que la formation Analytics Engineering remplira par « dbt build ».
    dbt_build = EmptyOperator(task_id="dbt_build")

    bronze >> raw >> corpus >> dbt_build


boreal_quotidien()
```

8.3 Attendre que le lecteur de DAG trouve le fichier : il cherche les fichiers neufs toutes les 5 minutes. Fenêtre 2 :

```
docker compose exec airflow-scheduler airflow dags list
# Attendu : une ligne boreal_quotidien. Pas encore là : relancer une minute plus tard.
```

8.4 Vérifier qu'aucun DAG n'a échoué à la lecture. Fenêtre 2 :

```
docker compose exec airflow-scheduler airflow dags list-import-errors
# Attendu : No data found.
```



© 2026 iAlexMG. Tous droits réservés.

8.5 Lancer une exécution sur une journée qui a des données, le 15 mars 2026. Fenêtre 2 :

```
docker compose exec airflow-scheduler airflow dags trigger boreal_quotidien --logical-date 2026-03-15T00:00:00+00:00
# Attendu : une exécution manual__ suivie de l'heure du lancement, dont la logical_date vaut
# 2026-03-15, à l'état queued.
```

8.6 Suivre les exécutions jusqu'à ce qu'elles soient vertes, dans l'interface ou en ligne de commande. Fenêtre 2 :

```
docker compose exec airflow-scheduler airflow dags list-runs boreal_quotidien
# Attendu, en deux minutes environ : deux exécutions success, la manual__, dont la logical_date
# vaut 2026-03-15, et une scheduled__, celle de la veille, que le scheduler a créée seul.
# Encore running ou queued : relancer la commande.
```

## Étape 9 — Préparer un témoin, puis laisser le DAG mensuel rattraper

*Théorie : `Theorie_06_DE.md`, annexe A.9.*

9.1 Copier l'entrepôt : le témoin part du même état. Relancer d'abord la commande de 8.6 : les deux exécutions doivent être success. Une exécution encore queued écrirait après la copie, et la copie réussirait quand même, sans rien signaler. Fenêtre 1 :

```
copy plateforme\entrepot\boreal.duckdb plateforme\entrepot\temoin.duckdb
REM Attendu : 1 fichier(s) copié(s).
```

9.2 Charger les trois mois de 2026 d'un seul bloc, dans le témoin seulement. Fenêtre 1 :

```
set PIPELINE_CHEMINS_ENTREPOT=plateforme/entrepot/temoin.duckdb
python -m pipeline raw --livraisons erp,garanties --depuis 2026-01 --jusqua 2026-03
REM Attendu : fenêtre : du 2026-01-01 au 2026-03-31, RAW : 42 fichier(s) chargé(s) à ce passage,
REM puis étape raw (...) : fin, 20 664 lignes.
set PIPELINE_CHEMINS_ENTREPOT=
REM Attendu : rien. Le pipeline revient à boreal.duckdb.
```

9.3 Écrire le DAG mensuel. Dès que le lecteur de DAG le trouve, le scheduler crée ses trois exécutions, une par mois. Fenêtre 1 :

```
notepad 01_de\dags\boreal_mensuel.py
```

Accepter la création. Coller, enregistrer, fermer :

```
# -*- coding: utf-8 -*-
"""boreal_mensuel.py : les livraisons mensuelles de Boréal.

Une tâche, qui charge dans la RAW les extraits du mois de son intervalle de données : les neuf
fichiers de l'ERP et les cinq des garanties.
```



© 2026 iAlexMG. Tous droits réservés.

```
La règle qu'il illustre : le rattrapage. Mettez ce DAG en route avec trois mois de retard, et
le scheduler crée trois exécutions, une par mois, chacune avec son intervalle. Le résultat doit
être identique à un chargement des trois mois d'un seul bloc, et l'étape 10 le vérifie.

Pourquoi une seule tâche, alors que le quotidien en a quatre. Le mois n'apporte que des fichiers à
charger : le bronze de l'usine est quotidien, le corpus n'a pas de rythme, et l'extraction de l'ERP
par ODBC demande un pilote Windows, qui n'est pas dans l'image.
"""
from __future__ import annotations

from datetime import timedelta

import pendulum
from airflow.sdk import DeadlineAlert, DeadlineReference, SyncCallback, dag
from airflow.providers.standard.operators.bash import BashOperator
from airflow.timetables.interval import CronDataIntervalTimetable

from boreal_alertes import prevenir_retard

PIPELINE = "cd /projet/01_de && python -m pipeline"
# Le mois de l'exécution, en AAAA-MM, lu dans sa date logique : juste pour une exécution planifiée
# comme pour une exécution lancée à la main. La fenêtre accepte cette forme, et une borne de mois
# vaut tout le mois. Écrire les deux bornes laisse le DAG lisible quand on voudra un trimestre.
MOIS = "{{ logical_date.strftime('%Y-%m') }}"


@dag(
    dag_id="boreal_mensuel",
    # Une tranche par mois, en toutes lettres, comme dans le quotidien. L'exécution de janvier part
    # le 1er février, une fois le mois fini.
    schedule=CronDataIntervalTimetable("@monthly", timezone="UTC"),
    start_date=pendulum.datetime(2026, 1, 1, tz="UTC"),
    # La borne de fin est inclusive : mise au 2026-04-01, elle ferait naître une quatrième
    # exécution, celle d'avril. Elle ne planterait pas : le chargeur ne trouve aucun fichier dans
    # sa fenêtre, rend zéro ligne, et l'exécution passe au vert. Un mois vide déguisé en succès.
    end_date=pendulum.datetime(2026, 3, 31, tz="UTC"),
    catchup=True,
    max_active_runs=1,
    default_args={"retries": 1, "retry_delay": timedelta(minutes=5)},
    deadline=DeadlineAlert(
        reference=DeadlineReference.DAGRUN_QUEUED_AT,
        interval=timedelta(hours=2),
```



© 2026 iAlexMG. Tous droits réservés.

```
            callback=SyncCallback(
                prevenir_retard,
                kwargs={"message": f"boreal_mensuel du mois {MOIS} n'a pas fini en 2 heures"},
            ),
        ),
        tags=["boreal", "cold path"],
        doc_md=__doc__,
    )
    def boreal_mensuel():
        BashOperator(
            task_id="raw_du_mois",
            bash_command=f"{PIPELINE} raw --livraisons erp,garanties --depuis {MOIS} --jusqua {MOIS}",
            # Le même pool que le DAG quotidien : les deux écrivent le même fichier DuckDB.
            pool="entrepot",
        )


    boreal_mensuel()
```

9.4 Suivre le rattrapage, jusqu'à 5 minutes le temps que le DAG soit trouvé. Fenêtre 2 :

```
docker compose exec airflow-scheduler airflow dags list-runs boreal_mensuel
# Attendu : trois exécutions success, scheduled__2026-02-01, 2026-03-01 et 2026-04-01, pour les mois
# de janvier, février et mars. Encore running, ou rien : relancer la commande.
```

## Étape 10 — Prouver que le rattrapage égale le chargement d'un bloc

*Théorie : `Theorie_06_DE.md`, annexe A.10.*

10.1 Ouvrir l'interface web de DuckDB. Fenêtre 3 : une nouvelle Anaconda Prompt, dans `Projet_Boreal_intermediaire`, puis :

```
conda activate formations
duckdb -ui
```

Laisser cette fenêtre ouverte : la fermer arrête l'interface. Créer un notebook neuf (+ à côté de Notebooks) et le renommer « Labo 06 » (double-clic sur son titre, puis Entrée). Une requête par cellule, Ctrl+Entrée pour l'exécuter.

10.2 Attacher l'entrepôt rattrapé, en lecture seule.

```
-- Attendu : 0 rows returned.
ATTACH IF NOT EXISTS 'plateforme/entrepot/boreal.duckdb' AS rattrape (READ_ONLY);
```

10.3 Attacher le témoin, en lecture seule.



© 2026 iAlexMG. Tous droits réservés.

```
-- Attendu : 0 rows returned.
ATTACH IF NOT EXISTS 'plateforme/entrepot/temoin.duckdb' AS bloc (READ_ONLY);
```

10.4 Comparer une table, dans les deux sens, sans la colonne `_charge_le`.

```
-- Attendu : erp_vbak, 0 et 0.
SELECT 'erp_vbak' AS table_raw,
       (SELECT COUNT(*) FROM (FROM (SELECT * EXCLUDE (_charge_le) FROM rattrape.raw.erp_vbak)
                              EXCEPT FROM (SELECT * EXCLUDE (_charge_le) FROM bloc.raw.erp_vbak))) AS rattrape_sans_bloc,
       (SELECT COUNT(*) FROM (FROM (SELECT * EXCLUDE (_charge_le) FROM bloc.raw.erp_vbak)
                              EXCEPT FROM (SELECT * EXCLUDE (_charge_le) FROM rattrape.raw.erp_vbak))) AS bloc_sans_rattrape;
```

10.5 Détacher l'entrepôt rattrapé.

```
-- Attendu : 0 rows returned.
DETACH rattrape;
```

10.6 Détacher le témoin : le script qui suit les ouvre à son tour.

```
-- Attendu : 0 rows returned.
DETACH bloc;
```

10.7 Écrire le script qui compare les quatorze tables : le SQL ne sait pas boucler sur une liste de tables, le code oui. Fenêtre 1 :

```
notepad 01_de\comparer_rattrapage.py
```

Accepter la création. Coller, enregistrer, fermer :

```
# -*- coding: utf-8 -*-
"""comparer_rattrapage.py : la RAW rattrapée mois par mois contre la RAW chargée d'un bloc.

Les deux entrepôts partent du même état. L'un a reçu les trois mois de 2026 par trois exécutions du
DAG mensuel, une par mois ; l'autre les a reçus d'un seul bloc. Les quatorze tables que le DAG
mensuel possède doivent être identiques, à la colonne _charge_le près, qui porte l'heure du
chargement et qui diffère par construction.

Lancement, depuis Projet_Boreal_intermediaire, l'entrepôt libre : python 01_de/comparer_rattrapage.py
"""
from pathlib import Path

import duckdb

ENTREPOT = Path("plateforme/entrepot/boreal.duckdb")
TEMOIN = Path("plateforme/entrepot/temoin.duckdb")
POSSEDEES = ("erp_", "garanties_")
```



© 2026 iAlexMG. Tous droits réservés.

```
con = duckdb.connect()
con.execute(f"ATTACH '{ENTREPOT.as_posix()}' AS rattrape (READ_ONLY)")
con.execute(f"ATTACH '{TEMOIN.as_posix()}' AS bloc (READ_ONLY)")

tables = [t[0] for t in con.execute(
    "SELECT table_name FROM duckdb_tables() "
    "WHERE database_name = 'rattrape' AND schema_name = 'raw' ORDER BY table_name").fetchall()]
possedees = [t for t in tables if t.startswith(POSSEDEES)]

print(f"{len(possedees)} tables du DAG mensuel, comparées sans la colonne _charge_le\n")
print(f"{'table':<28}{'rattrapé':>10}{'bloc':>10}{'écarts':>9}")
ecarts = 0
for table in possedees:
    a = f"(SELECT * EXCLUDE (_charge_le) FROM rattrape.raw.{table})"
    b = f"(SELECT * EXCLUDE (_charge_le) FROM bloc.raw.{table})"
    n_a = con.execute(f"SELECT COUNT(*) FROM {a}").fetchone()[0]
    n_b = con.execute(f"SELECT COUNT(*) FROM {b}").fetchone()[0]
    # EXCEPT ALL, et non EXCEPT : le premier compte les exemplaires, le second compare des
    # ensembles et écarte les doublons avant de comparer. Un mois chargé deux fois serait
    # invisible à EXCEPT, puisque ses lignes en trop sont identiques à celles du témoin.
    manque = con.execute(f"SELECT COUNT(*) FROM (FROM {a} EXCEPT ALL FROM {b})").fetchone()[0]
    surplus = con.execute(f"SELECT COUNT(*) FROM (FROM {b} EXCEPT ALL FROM {a})").fetchone()[0]
    # L'écart de comptes entre dans le verdict : une preuve qui ne peut pas échouer ne prouve rien.
    ecart_table = manque + surplus + abs(n_a - n_b)
    ecarts += ecart_table
    print(f"{table:<28}{n_a:>10}{n_b:>10}{ecart_table:>9}")

con.close()
print(f"\nVerdict : {'0 écart, le rattrapage égale le bloc' if ecarts == 0 else f'{ecarts} écarts'}")
```

10.8 Lancer la comparaison. Fenêtre 1 :

```
python 01_de\comparer_rattrapage.py
REM Attendu : 14 tables, de erp_cdhdr à garanties_visites, 0 écart chacune,
REM puis Verdict : 0 écart, le rattrapage égale le bloc.
```

10.9 Effacer le témoin : il a fait son travail. Fenêtre 1 :

```
del plateforme\entrepot\temoin.duckdb
REM Attendu : rien.
```



© 2026 iAlexMG. Tous droits réservés.

## Étape 11 — Versionner la fenêtre, les DAG et les tests

*Théorie : `Theorie_06_DE.md`, annexe A.11.*

Fenêtre 1. `git -C 01_de` lance Git dans le dépôt sans quitter `Projet_Boreal_intermediaire`, et les chemins qui suivent partent de `01_de` :

```
git -C 01_de add dags pipeline tests comparer_rattrapage.py
REM Attendu : rien.
git -C 01_de status
REM Attendu : Changes to be committed, cinq fichiers en new file (les trois du dossier dags,
REM fenetre.py et comparer_rattrapage.py) et sept en modified ; rien de la plateforme.
git -C 01_de commit -m "Cours 06 : la fenetre du pipeline et les deux DAG"
REM Attendu : 12 files changed.
git -C 01_de status
REM Attendu : nothing to commit, working tree clean.
```

Pour tout fermer : `.exit` à l'invite `D` de la fenêtre 3, puis fermer l'onglet du navigateur. La pile peut s'éteindre sans rien perdre, sans quoi le DAG quotidien tourne chaque nuit. Fenêtre 2 :

```
docker compose stop
# Attendu : chaque service Stopped. La commande de démarrage de 6.1 la rallume, données comprises.
```

---

## Dépannage

| **Symptôme** | **Correctif** |
|---|---|
| `No module named pipeline` | la fenêtre 1 n'a pas son `set PYTHONPATH=01_de` (fenêtre rouverte) : le retaper, puis relancer |
| `python: can't open file '…\_univers\boreal\generer_boreal.py'` | la fenêtre 1 n'est pas dans `Projet_Boreal_intermediaire` : la rouvrir comme au début |
| Le Bloc-notes ouvre un fichier qui a déjà du contenu en 2.1, 8.1, 8.2, 9.3 ou 10.7 | un reste d'un premier essai : tout sélectionner, coller le fichier à sa place, enregistrer |
| `No module named 'pipeline.fenetre'` | le Bloc-notes a enregistré `fenetre.py.txt` (`dir 01_de\pipeline` le montre) : `ren 01_de\pipeline\fenetre.py.txt fenetre.py` |
| `no configuration file provided: not found` | la fenêtre 2 n'est pas dans `plateforme` : la fermer, puis la rouvrir comme au début |



© 2026 iAlexMG. Tous droits réservés.

| Symptôme | Correctif |
|---|---|
| `docker compose ps` montre `series`, `mqtt`, `kafka` ou `grafana` | un `docker compose up -d` lancé sans nommer de service : ces services servent aux Cours 08 à 12, `docker compose stop series mqtt kafka grafana` |
| l'interface répond `502` sur `localhost:8080` | le serveur d'API n'a pas fini de démarrer : attendre que `docker compose ps` le dise `(healthy)` |
| le DAG n'apparaît pas dans `airflow dags list` | un fichier neuf que le lecteur de DAG n'a pas encore trouvé : attendre jusqu'à 5 minutes ; si rien ne vient, `airflow dags list-import-errors` nomme le fichier et la ligne fautive |
| le même DAG sur plusieurs lignes de `airflow dags list` | une ligne par version du DAG, chaque changement de structure du fichier en crée une : rien à corriger |
| `Provided callback ... is not awaitable` | `AsyncCallback` reçoit une fonction ordinaire : `SyncCallback`, comme dans le labo |
| `dags trigger` plante sur `UniqueViolation` et `dag_run_dag_id_logical_date_key` | cette date a déjà une exécution dans ce DAG : `airflow dags delete boreal_quotidien -y`, attendre qu'il revienne dans `airflow dags list`, puis relancer |
| `DagNotFound: Dag id ... not found in DagModel` juste après `dags delete` | le lecteur de DAG n'a pas encore relu le fichier : attendre une demi-minute |
| `airflow dags list-runs -d boreal_mensuel` répond `unrecognized arguments: -d` | la ligne de commande a changé en Airflow 3 : le DAG se donne en argument, sans `-d` |
| `airflow tasks clear` s'arrête sur `TypeError: unsupported type for timedelta seconds component: dict` | une limite d'Airflow 3.3.1 sur les DAG qui portent une Deadline Alert : `airflow dags delete <dag_id> -y`, puis relancer |
| une exécution met cinq minutes de plus que prévu | une tâche a raté une fois, et la nouvelle tentative a attendu son délai : le journal de la tâche, dans l'interface, dit pourquoi |
| `Could not set lock on file ... Conflicting lock is held` dans une tâche | un autre programme tient l'entrepôt, l'interface de DuckDB par exemple : `DETACH`, et la nouvelle tentative passera |
| `copy` du témoin : `Le processus ne peut pas accéder au fichier` | une tâche d'Airflow écrit l'entrepôt : attendre que 8.6 montre les exécutions `success`, puis recopier |



© 2026 iAlexMG. Tous droits réservés.